MUSIC PLAYER

A high-performance, C-based command-line music engine built on three complementary data structures — designed for speed, memory efficiency, and a seamless listening experience.

C SYSTEMS PROGRAMMING | DATA STRUCTURES

Presenters

Rafsun Jani - 253-15-409
Somaiya Akter - 253-15-788
Lamyea Luice Laisha - 253-15-225
Shaker Abdullah - 253-15-865
Mst. Tasnima Khatun - 253-15-949

Vision & Core Features

Our mission is to engineer a **high-performance, C-based music streaming engine** that delivers a full-featured listening experience entirely within a command-line interface. By leveraging carefully chosen data structures, the engine achieves algorithmic efficiency without sacrificing usability. Every architectural decision is grounded in measurable performance criteria — search speed, memory footprint, and navigation latency — ensuring the system scales gracefully as library sizes grow.

Smart Search

Binary Search Tree–powered track lookup delivers **O(log N)** search complexity, enabling instant retrieval even from libraries containing tens of thousands of songs. Users can search by title, artist, or album with near-zero latency.

Bidirectional Navigation

A Doubly Linked List enables **O(1) constant-time** forward and backward track transitions. No array reshuffling, no index recalculation — just direct pointer arithmetic for instantaneous playback control.

Loop Mode

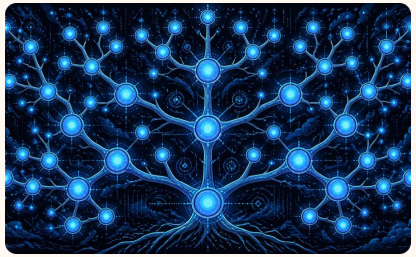
By dynamically transforming the Doubly Linked List into a Circular Linked List at runtime, repeat mode activates without duplicating any track data in memory — a clean, zero-overhead implementation.

LIFO Play History

A Singly Linked Stack automatically logs every completed track using Last-In-First-Out semantics, enabling instant "previously played" recall and a reliable undo mechanism for accidental skips.

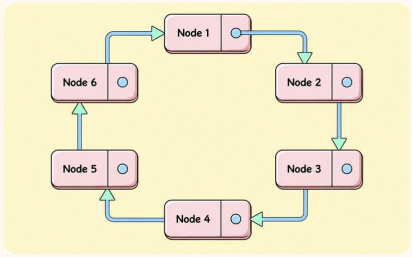
Three Data Structures, One Unified Engine

The architecture rests on three purpose-built data structures, each selected to solve a distinct class of problem within the music player. Together, they form a cohesive system where each module handles its domain with optimal algorithmic complexity. The **MusicPlayer** master controller acts as the central dashboard, orchestrating data flow between all three modules seamlessly.



1. BST — Search Engine

A Binary Search Tree organizes the entire track library for **O(log N) lookup**. Supports recursive insertion, balanced search, and in-order traversal for a naturally sorted library view. The primary interface for track discovery and retrieval.



2. Doubly / Circular Linked List — Playback

A Doubly Linked List manages the active playback queue with **O(1) next/previous navigation**. Seamlessly transforms into a Circular Linked List for loop mode by linking tail to head — no memory duplication required.

3. Stack — History Tracking

A Singly Linked Stack implements **LIFO semantics** for play history. Tracks are auto-pushed on completion and popped on demand, providing a lightweight, memory-efficient undo layer for the playback session.

MusicPlayer Master Controller: A central struct that holds pointers to the BST root, DLL head/tail, and Stack top — unifying all three modules under a single API surface and enabling clean cross-module data handoff.

Module 1: The BST Search Engine

BINARY SEARCH TREE

Core Functions

Recursive Insertion

New tracks are inserted by comparing sort keys (e.g., track title) at each node.

The function recurses left or right until an empty slot is found, maintaining the BST ordering invariant automatically.

Recursive Search

Search compares the target key against the current node, recursing into the matching subtree.

Returns a pointer to the track node on success, or `NULL` on miss.

In-Order Traversal

Visits nodes in ascending sorted order (**Left → Root → Right**).

Produces a naturally sorted library view without any additional sorting step — ideal for displaying the full track list.

Module 2: Sequential Playback Controller

DOUBLY LINKED LIST | O(1) NAVIGATION

The playback queue is managed by a **Doubly Linked List (DLL)**, where each node represents a track and stores pointers to both its predecessor and successor. This dual-pointer design is the key enabler of seamless bidirectional navigation — the user can move forward or backward through the queue with equal efficiency, regardless of queue length.

Why a Doubly Linked List?

Bidirectional Navigation

Dual pointers (prev and next) allow traversing the queue forward and backward with equal efficiency, eliminating the need to re-index or restart traversal from the beginning.

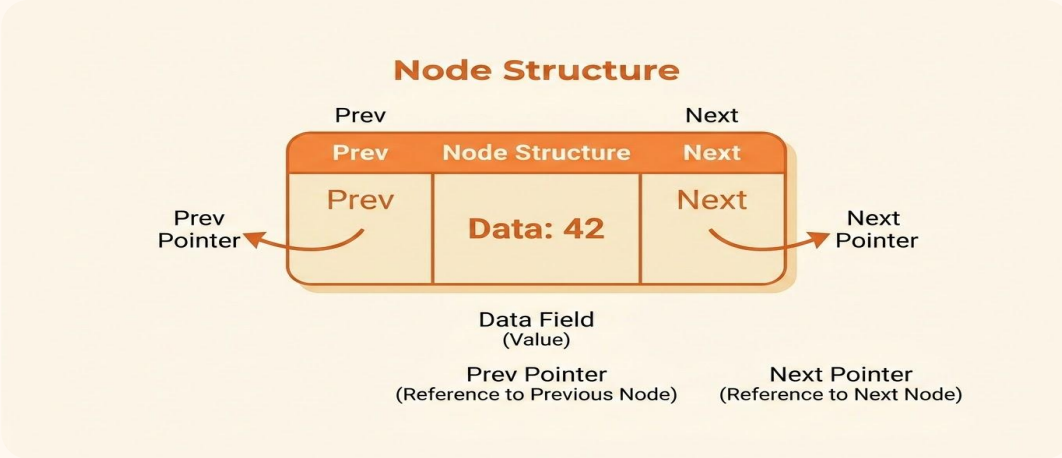
O(1) Time Complexity

Inserting or deleting a track at a known position only requires updating neighboring pointers. Unlike static arrays, no subsequent elements need to be shifted in memory.

Dynamic Memory Scaling

Memory is allocated dynamically on a per-node basis. The queue grows and shrinks fluidly with the playlist size, avoiding the wasted overhead of pre-allocated array buffers.

Node Structure



Feature Spotlight: Loop Mode

DLL → CLL TRANSFORMATION

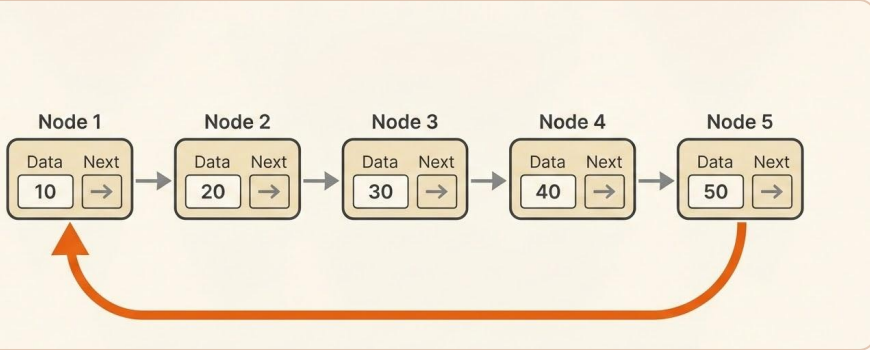
ZERO MEMORY OVERHEAD

Loop mode — the ability to repeat a playlist continuously — is implemented not by duplicating data or re-enqueueing tracks, but by a **single pointer reassignment** that transforms the Doubly Linked List into a Circular Linked List (CLL). This is one of the most elegant demonstrations of how data structure choice directly enables feature design.

The Transformation Logic

When loop mode is activated, the engine performs a single link update that connects the end of the playlist back to the beginning.

This closes the list into a ring, causing playback to wrap around seamlessly without duplication.



Behavioral Modes

Normal Mode

The linked list operates linearly. Playback stops at the final track.

Loop All

Circular mode is active. The entire queue repeats indefinitely.

Loop One

Special case: a single track repeats until the mode is toggled off.

The same underlying structure supports all three modes — only the pointer configuration changes. This demonstrates the power of choosing a flexible, pointer-based structure over a rigid array.

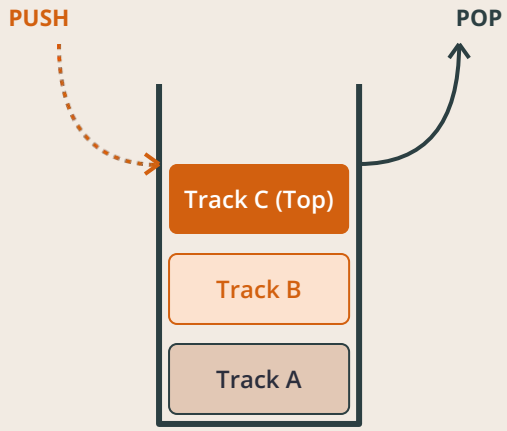
Module 3: Play History & Memory Tracking

LIFO STACK | SINGLY LINKED IMPLEMENTATION

Play history is a critical UX feature — users expect to recall what they just listened to, and to undo accidental skips. The **Last-In-First-Out (LIFO)** access pattern of a stack maps perfectly to this requirement: the most recently played track is always the first one retrieved. A Singly Linked Stack provides this with minimal memory overhead and O(1) push/pop operations.

Stack Architecture

The history stack stores lightweight track metadata such as the track title and artist rather than full file paths, keeping each entry compact. A count field enables bounded history, so the stack can be capped at a fixed number of entries to prevent unbounded memory growth during long sessions.



Automatic Push Trigger Mechanism

Track completion events automatically trigger a stack push — no manual intervention required from the user or the playback controller's main loop:

01 **Track Finishes Playing**
The playback engine signals completion of the current track's audio output.

02 **Push Event Fires**
A new history entry is allocated and pushed onto the stack top in constant time.

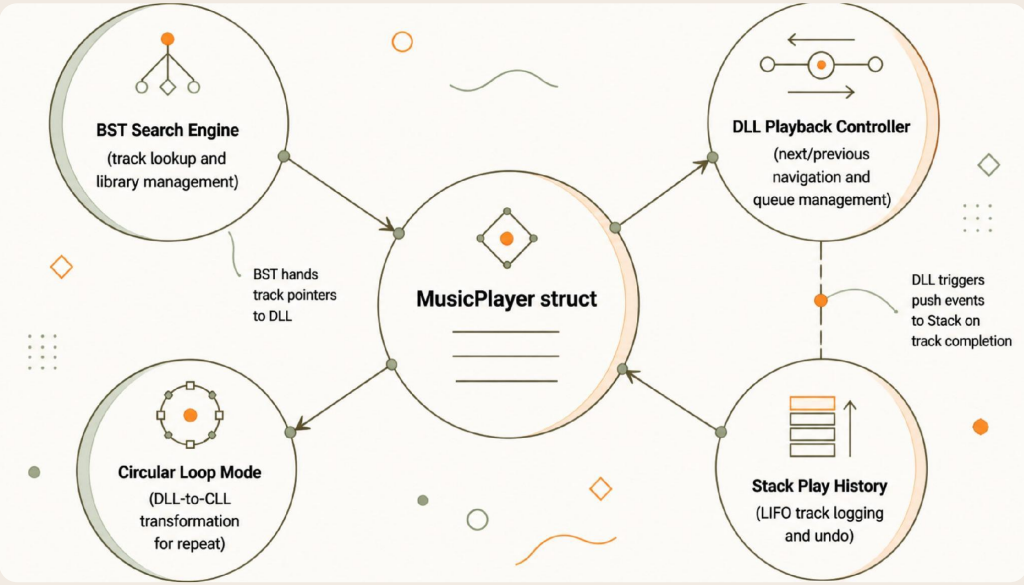
03 **Transition Occurs**
Playback advances to the next track via the linked list. History is now updated and ready for recall.

04 **User Requests History**
A pop operation retrieves the most recent track for re-queuing or display.

System Integration: The Master Controller

MUSICPLAYER STRUCT | CROSS-MODULE ORCHESTRATION

System Architecture Diagram



Music Player Struct Hub

The central controller coordinates search, playback, repeat modes, and history via lightweight pointer operations without data duplication.

Cross-Module Data Handoff

One of the most important integration patterns is the **BST → DLL handoff**. When a user searches for and selects a track:

01

Search Returns Track Pointer

The search module returns the selected track directly, with no duplication or extra copying.

02

Pointer Injected into Playback

The same track reference is placed into the playback queue or set as the active selection for immediate playback.

03

Zero-Copy Architecture

Track data lives in one shared structure, reducing overhead and keeping the modules synchronized.

Technical Feasibility: Hybrid vs. Array

PERFORMANCE COMPARISON | ARCHITECTURE JUSTIFICATION

The hybrid architecture — BST + DLL + Stack — delivers better performance than a simple or dynamic array across the operations that matter most in a music engine.

Operation	Array	Hybrid Design	Advantage
Track Search	Linear scan	Logarithmic search	~700× faster
Mid-Queue Insert	Shift / resize	Relink pointers	Dramatic
Loop Mode	Re-enqueue	Pointer swap	Zero-copy
Sorted Library	Sort every time	In-order traversal	Always sorted

700×

Search Speedup

Hybrid search vs. linear array search at 10,000 tracks.

0

Memory Copies

Loop mode activation uses pointer swap only.

3

Structures

BST, DLL, and Stack unified in one engine.

O(1)

Navigation Time

Constant-time next, previous, push, and pop.

Questions & Answers